
SISTEMA DE CATÁLOGO DE BIBLIOTECA CON ESTRUCTURAS DE DATOS PROPIAS EN C#

Carnet 202504845 – Ixchel Panjoj García

Resumen

Este ensayo presenta el desarrollo de un catálogo de biblioteca hecho en C#, creado como proyecto del curso Introducción a la Programación y Computación 2. El sistema permite guardar libros, organizarlos en categorías y subcategorías, y mostrar esa organización de forma visual. Lo particular del proyecto es que no se usó ninguna herramienta de almacenamiento ya construida en el lenguaje: cada estructura para guardar y ordenar los datos se programó desde cero, entendiendo cómo funciona por dentro. Se explican las decisiones tomadas para guardar los libros de forma ordenada y rápida de buscar, y para representar categorías que pueden tener muchos niveles, como carpetas dentro de carpetas. También se describe cómo se conectó esa lógica con una interfaz web sencilla, y algunos problemas reales que aparecieron durante las pruebas, junto con la forma en que se corrigieron. El objetivo es mostrar que construir estas herramientas propias, aunque toma más tiempo, ayuda a comprender mejor cómo funcionan los programas.

Palabras clave

árboles balanceados, catálogo de biblioteca, estructuras de datos, programación en C#, aplicación web

Abstract

This essay presents the development of a library catalog application built in C#, created as a project for the course Introduction to Programming and Computing 2. The system stores books, organizes them into categories and subcategories, and displays that organization visually. What makes the project particular is that no built-in storage tool from the language was used: every structure for storing and ordering data was written from scratch, understanding how it works internally. The essay explains the decisions made to keep books stored in an order that is fast to search, and to represent categories that can have many levels, like folders inside folders. It also describes how that logic was connected to a simple web interface, and some real problems that came up during testing, along with how they were solved. The goal is to show that building these tools from scratch, although it takes more time, helps in understanding better how programs that are usually taken for granted actually work.

Keywords

balanced trees, library catalog, data structures, C# programming, web application

Introducción

Cuando se programa, es común usar herramientas ya construidas para guardar y organizar información, como listas o diccionarios que el lenguaje ofrece. En este proyecto se construyeron esas herramientas desde cero, para un catálogo de biblioteca que debe guardar libros, agruparlos por categorías y mostrar esa organización de forma visual. El resultado se vuelve interesante cuando el catálogo crece, porque buscar un libro o mostrar una jerarquía de categorías con muchos niveles deja de ser trivial si no se elige bien la forma de guardar los datos. Este ensayo explica qué estructuras se eligieron y por qué, cómo se integraron con una interfaz web, y qué problemas reales aparecieron al probar el sistema con datos más grandes, y cómo se solucionaron.

Desarrollo del tema

El desarrollo del catálogo se puede explicar en cuatro partes: la razón para construir las estructuras propias, la forma en que se organiza la información, la arquitectura general de la aplicación, y los problemas encontrados durante las pruebas.

a. Por qué construir las estructuras propias

El curso pide explícitamente no usar las herramientas de almacenamiento que ya vienen con C#, como listas o diccionarios genéricos. La razón detrás de esta regla es que, al usar una herramienta ya hecha, no se entiende realmente cómo guarda los datos por dentro ni por qué es rápida o lenta en ciertos casos. Construir

cada estructura a mano obliga a decidir, por ejemplo, cómo se compara un libro con otro para ordenarlo, o qué se hace cuando una categoría no tiene todavía ninguna subcategoría. Ese proceso de decisión es, en sí mismo, la parte más formativa del proyecto. Por ejemplo, la regla obligó a decidir cómo comparar dos libros para poder ordenarlos: se eligió comparar directamente el número de ISBN, ya que es un valor único para cada libro y no cambia con el tiempo. También obligó a decidir qué hacer cuando se elimina el único libro de una categoría: la categoría no desaparece, simplemente queda con su lista de libros vacía, lista para recibir libros nuevos más adelante. Decisiones como estas no se notan cuando se usa una herramienta ya construida, porque esa herramienta ya las tomó por el programador.

b. Cómo se organiza la información

Los libros se guardan en un árbol donde cada libro se compara con otros según su ISBN, de manera que buscar, agregar o quitar un libro no requiere revisar todos los libros uno por uno. Este árbol se mantiene balanceado, lo que significa que, después de cada cambio, se revisa que ningún lado del árbol quede mucho más alto que el otro, y si eso ocurre, se reacomoda. Sin ese balance, si los libros se agregan casi en orden, el árbol podría terminar pareciendo una lista larga y perdería la ventaja de ser rápido. Para las categorías se usó una idea distinta, porque una categoría puede tener varias subcategorías y no solo dos: cada categoría guarda solamente un enlace a su primera subcategoría y un enlace a la siguiente categoría del mismo nivel, y con esos dos enlaces es posible recorrer cualquier cantidad de niveles y de categorías hermanas sin necesitar una lista aparte para cada una. Por ejemplo, si los libros se insertaran siempre en orden creciente de ISBN sin balancear el

árbol, cada libro nuevo terminaría colgado del lado derecho del anterior, y el árbol se comportaría igual que una lista: buscar el último libro insertado tomaría revisar todos los demás. Al mantenerlo balanceado, aunque se inserten cien libros en orden, la altura del árbol crece muy poco, y cualquier búsqueda revisa solamente unos pocos niveles en lugar de recorrer todo el catálogo.

c. Arquitectura de la aplicación

El proyecto se dividió en tres partes. La primera contiene únicamente la lógica de datos: los árboles, las listas propias y las reglas para agregar o eliminar libros y categorías, sin saber nada sobre cómo se va a mostrar esa información. La segunda parte es la interfaz web, construida con Blazor, que muestra formularios para cargar el archivo inicial, registrar libros y armar el árbol de categorías, y que llama a la primera parte cada vez que el usuario hace una acción. La tercera parte se encarga de generar una representación visual de la jerarquía de categorías y de los libros, usando el lenguaje Graphviz, que se dibuja directamente en el navegador. Separar estas tres partes permitió corregir errores en la lógica de datos sin tener que tocar la interfaz, y probar cada parte por separado. La Figura 1 muestra cómo se relacionan las clases principales del sistema. Dentro de la interfaz web, cada página de Blazor obtiene el catálogo mediante inyección de dependencias, por lo que todas las páginas comparten la misma instancia mientras la aplicación sigue corriendo, sin necesidad de volver a cargar el archivo XML entre una pantalla y otra. Para mostrar el árbol de categorías, que puede tener una cantidad variable de niveles y de subcategorías, se usó un componente de Blazor que se llama a sí mismo: cada categoría se dibuja y, por cada una de sus subcategorías, el mismo componente

se vuelve a usar para dibujar un nivel más abajo. Esta técnica evita tener que escribir un bloque de código distinto para cada nivel posible de la jerarquía.

d. Problemas encontrados y cómo se resolvieron

Durante las pruebas con un catálogo más grande aparecieron dos errores reales. El primero ocurría al agregar una segunda categoría principal: la lógica guardaba la nueva categoría en el lugar de la anterior en lugar de agregarla junto a ella, por lo que solo la última categoría principal quedaba visible y las demás desaparecían del árbol. Se corrigió haciendo que cada categoría nueva se enlace correctamente junto a las que ya existían, en vez de reemplazarlas. El segundo error aparecía al eliminar un libro: la lista de libros de una categoría no actualizaba correctamente su último elemento ni la cantidad de libros que contenía, lo que podía provocar datos inconsistentes en operaciones posteriores. Ambos errores se detectaron gracias a pruebas con casos variados, como catálogos vacíos, categorías repetidas y búsquedas de libros inexistentes, lo que confirma la importancia de probar no solo los casos esperados, sino también los casos límite.

Resultados de las pruebas

Antes de dar por terminado el proyecto se probó el sistema con casos límite: un catálogo vacío, la búsqueda de un ISBN que no existe, el intento de registrar dos veces la misma categoría, la eliminación de un ISBN que no está en el catálogo, y el intento de crear una subcategoría bajo una categoría padre que no existe. En todos los casos el sistema respondió con un mensaje adecuado en lugar de fallar o mostrar información incorrecta. También se probó con un archivo más grande, con quince libros distribuidos en nueve categorías organizadas en tres niveles, por ejemplo Ficción con la subcategoría Fantasía y,

dentro de esta, Fantasía Épica. La visualización con Graphviz mostró correctamente los tres niveles de anidamiento y agrupó cada libro bajo la categoría exacta a la que pertenece, sin mezclarlo con las categorías padre ni con las categorías hermanas. Estas pruebas con más datos fueron precisamente las que permitieron encontrar los dos errores descritos anteriormente, ya que con pocos datos ambos problemas pasaban desapercibidos.

Todas estas pruebas se realizaron directamente sobre la interfaz web, cargando distintos archivos de entrada y revisando en pantalla que cada operación mostrara el resultado esperado, en lugar de asumir que el código era correcto solo porque compilaba sin errores. Cada corrección se probó de nuevo antes de continuar con la siguiente parte del proyecto, lo que evitó que un error ya corregido volviera a aparecer más adelante sin darse cuenta.

Descripción de las estructuras utilizadas

A continuación se describe brevemente cada una de las estructuras propias que forman el sistema.

ArbolLibros. ArbolLibros es un árbol AVL que guarda todos los libros del catálogo ordenados por su ISBN. Cada nodo del árbol conoce su propia altura, lo que permite detectar cuándo un lado quedó más alto que el otro después de insertar o eliminar un libro, y reacomodar el árbol cuando es necesario. Gracias a este balance, buscar un libro, encontrar el de ISBN menor o el de ISBN mayor, o recorrer todos los libros en orden son operaciones rápidas incluso si el catálogo crece bastante.

ArbolCategorias. ArbolCategorias guarda, por un lado, el árbol jerárquico de categorías y subcategorías, y por otro lado, un árbol de búsqueda

auxiliar que asocia cada nombre de categoría con el nodo correspondiente dentro de esa jerarquía. Este segundo árbol existe únicamente para poder encontrar una categoría por su nombre sin tener que recorrer toda la jerarquía desde la raíz cada vez que se necesita.

NodoCategoria. NodoCategoria representa una sola categoría dentro de la jerarquía. En lugar de guardar una lista de subcategorías, cada nodo guarda solamente dos enlaces: uno hacia su primera subcategoría y otro hacia la siguiente categoría del mismo nivel. A partir de esos dos enlaces es posible llegar a cualquier subcategoría y a cualquier categoría hermana, sin importar cuántas existan.

ListaLibros. ListaLibros es una lista enlazada propia que se utiliza en dos lugares distintos: dentro de cada categoría, para guardar los libros que pertenecen a ella, y como resultado de los recorridos del árbol de libros. Al ser una sola implementación reutilizada en ambos casos, no fue necesario escribir dos estructuras distintas para resolver el mismo problema.

CatalogoLibreria. CatalogoLibreria es la clase que conecta las demás estructuras y expone las operaciones que la interfaz web necesita, como registrar o eliminar un libro, registrar una categoría o una subcategoría, y obtener los libros de una categoría ya ordenados. Esta clase es la única que la interfaz conoce directamente; todo el resto de la lógica de datos queda oculto detrás de ella.

Conclusiones

Construir las estructuras de datos desde cero, en lugar de usar las que ya trae el lenguaje, permitió entender con claridad por qué el balance de un árbol y la forma de enlazar los nodos afectan directamente qué tan

rápido funciona un programa cuando la cantidad de datos crece.

Representar una jerarquía de categorías con varios niveles no necesita una estructura distinta para cada nivel: basta con que cada nodo sepa cuál es su primera subcategoría y cuál es la siguiente categoría del mismo nivel para poder recorrer cualquier cantidad de niveles y de categorías hermanas.

Separar la lógica de datos de la interfaz de usuario facilitó encontrar y corregir errores, porque cada parte se pudo probar por separado, y los dos problemas reales encontrados durante las pruebas se detectaron precisamente gracias a esa separación.

Probar el sistema con datos vacíos, datos repetidos y datos más grandes de lo normal resultó tan importante como escribir la lógica principal, porque los dos errores reales del proyecto solamente aparecieron cuando el catálogo dejó de ser pequeño.

Como trabajo futuro, estas mismas estructuras podrían extenderse para permitir que un libro pertenezca a más de una categoría a la vez, o para guardar el catálogo en un archivo en lugar de perderlo cada vez que la aplicación se reinicia.

Joyanes Aguilar, L., y Zahonero Martínez, I. (2008). Estructuras de datos en Java. McGraw-Hill.

Microsoft. (2025). ASP.NET Core Blazor overview. Microsoft Learn.

<https://learn.microsoft.com/aspnet/core/blazor/>

Referencias bibliográficas

Cormen, T. H., Leiserson, C. E., Rivest, R. L., y Stein, C. (2009). Introduction to algorithms (3.a ed.). MIT Press.

Graphviz. (2024). Graphviz documentation. <https://graphviz.org/documentation/>

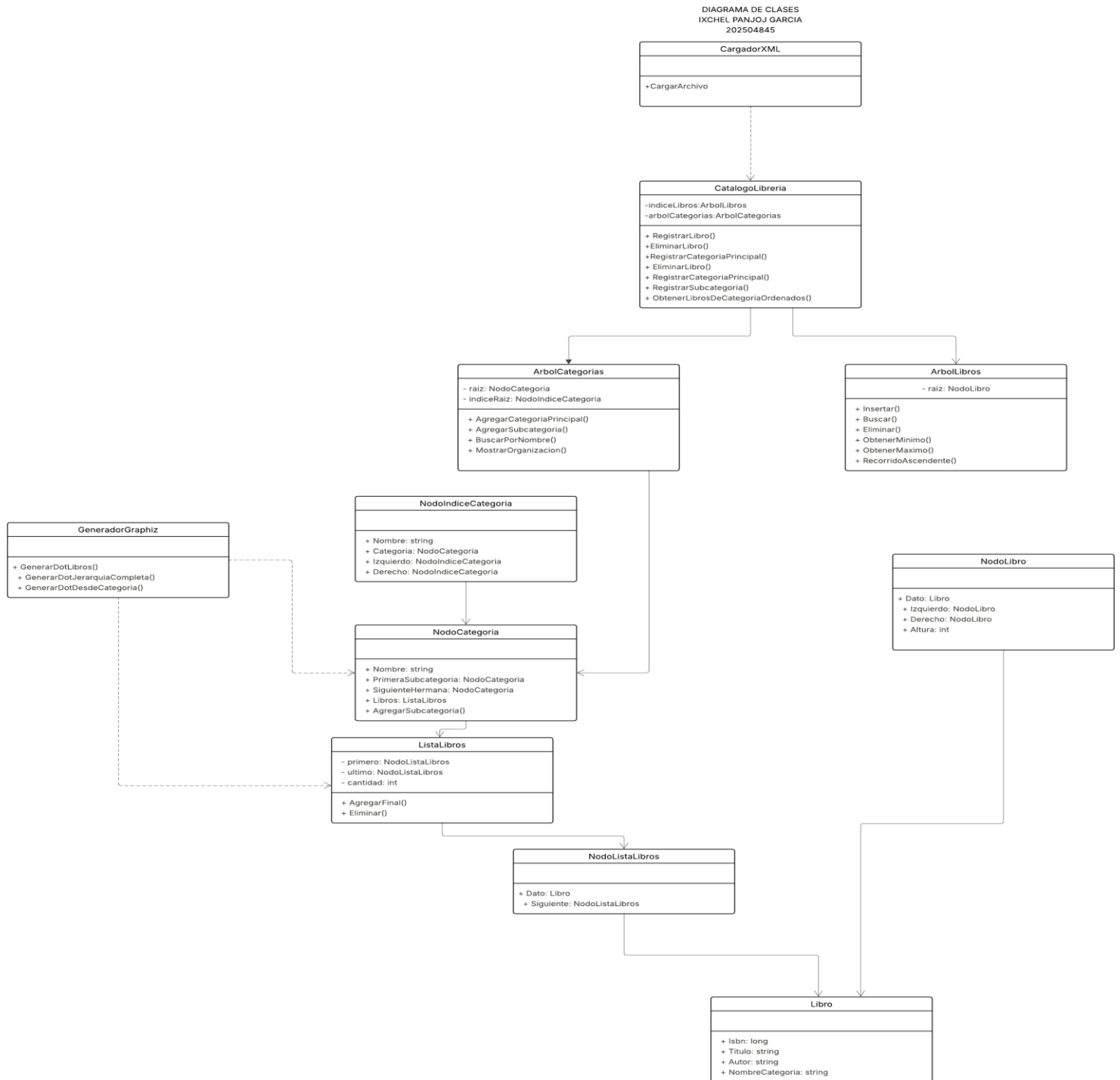


Figura 1. Diagrama de clases del sistema.

Fuente: elaboración propia.